

Trees and Heaps

Weike Fang

1 Introduction

In this note we introduce two fundamental data structures that appear throughout data structures and algorithms: *trees* and *heaps*. We will first define general tree terminology, then focus on a special class of trees (called *complete* trees) that are used to implement heaps efficiently. Finally, we will see how heaps can be stored in arrays using simple index formulas.

2 What is a Tree?

Intuitively, a **tree** is a hierarchical data structure used to represent data in a parent–child relationship. Each element in the tree is called a *node*, and nodes are connected by *edges*.

Formally, a tree can be defined as a connected, acyclic graph that satisfies:

- There is a distinguished **root** node at the top.
- Every node (except the root) has exactly one parent.
- There is exactly one simple path between any pair of nodes.

Two common special cases are:

- **Binary tree**: each node has at most two children.
- **d -ary tree**: each node has at most d children.

3 Tree Terminology

Given a node i in a tree, we use the following terminology:

- **Parent(i)**: the node directly above i that is connected to i by an edge.
- **Child(i)**: a node directly below i that has i as its parent.
- **Siblings**: nodes that share the same parent.
- **Root**: the unique node with no parent.
- **Leaf**: a node with zero children.
- **Subtree(n)**: the tree consisting of node n and all of its descendants.

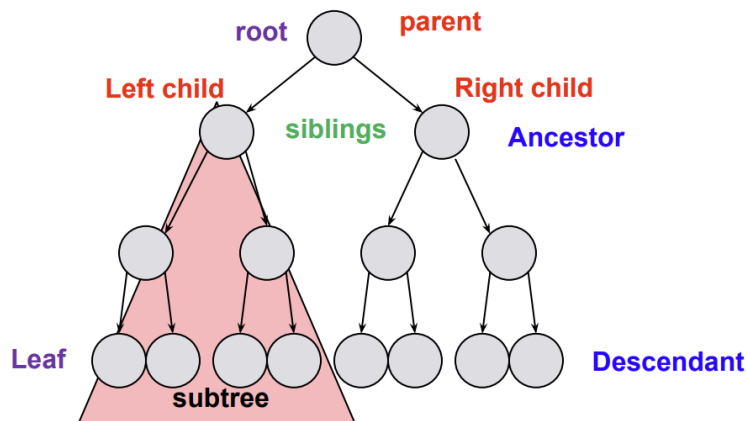


Figure 1: Tree terminology: root, parent/child, siblings, leaves, and subtrees.

4 Full and Complete d -ary Trees

For a d -ary tree (where each node has at most d children), we often care about how “packed” the tree is.

Full d -ary tree: A d -ary tree where every node has either 0 or d children and **all leaf nodes are at the same level**. In such a tree, adding one more node requires increasing the height of the tree.

Complete d -ary tree: A d -ary tree where:

- All levels except possibly the last are completely full.
- Nodes on the last level are filled from left to right with no gaps.

Complete binary trees are especially important because they can be stored very efficiently in arrays, which is exactly what we will use for heaps.

5 What is a Heap?

A **heap** is a tree that satisfies two key properties:

1. It is a **complete** d -ary tree (usually a complete binary tree).
2. It satisfies the **heap property**: every parent is “better” than its children, where “better” depends on whether we use a minimum or maximum ordering.

There are two common variants:

- **Min-heap**: every node is less than or equal to all of its children.
- **Max-heap**: every node is greater than or equal to all of its children.

Note that the heap property is *local*: it relates a node only to its children. A min-heap does not require that left child $\leq$ right child, and it does not require that nodes on the same level are sorted.

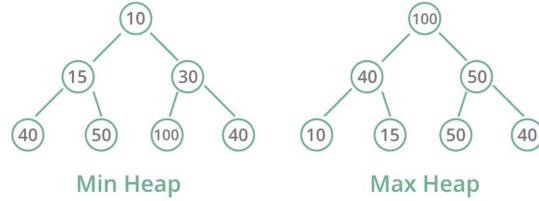


Figure 2: Example of a min-heap (left) and a max-heap (right).

Min-heap vs. Max-heap Example

In the min-heap example (Figure 2), every parent node has a value less than or equal to each of its children. In the max-heap example, each parent value is greater than or equal to its children.

6 Storing a Heap in an Array

Because a heap is a complete binary tree, we can store it compactly in an array by listing the nodes in *level-order* (top to bottom, left to right). This eliminates the need for explicit pointers between nodes; instead, we use simple index arithmetic to move between a node and its parent/children. We will look at two common conventions:

6.1 Array indices starting at 1

Assume the array index 1 stores the root of the heap and index 0 is unused. Then for a node at index i :

$$\begin{aligned}\text{Parent}(i) &= \left\lfloor \frac{i}{2} \right\rfloor, \\ \text{Left_child}(i) &= 2i, \\ \text{Right_child}(i) &= 2i + 1.\end{aligned}$$

6.2 Array indices starting at 0

In many programming languages (e.g., C++ and Java), arrays are 0-indexed. If we store the root at index 0, then for a node at index i :

$$\begin{aligned}\text{Parent}(i) &= \left\lfloor \frac{i-1}{2} \right\rfloor \quad (i > 0), \\ \text{Left_child}(i) &= 2i + 1, \\ \text{Right_child}(i) &= 2i + 2.\end{aligned}$$